Auto Repair Database Summary Report

Summary

This project aims to address the challenge of data keeping in the automobile repair industry. Often, this industry relies heavily on keeping track of information by hand, which has benefits, but in the modern age, this can be unorganized, take up a lot of time, and lead to poor business decisions. A simple solution to this issue is for automobile repair shops to utilize a computerized database which can keep track of all the fundamental data pieces which an automobile repair shop can thrive on. In the case of this specific project, this was done using PostgreSQL, with various other supporting tools to help create a structured database, alongside valuable SQL queries to support key business decisions. The project utilizes a Kaggle dataset, linked here, which contains motor vehicle data from 2020 to 2024. This data was imported, and through the use of queries, this data could be manipulated in various ways, such as through data retrieval, filtering, aggregation, subqueries, and quality validation. This ultimately leads to fascinating insights toward things such as most frequent repair types, seasonal service volume trends, customer patterns, and data integrity, which are all extremely valuable when it comes to making operational decisions and thriving as an auto repair business.

Introduction

Purpose of the Project

The purpose of this project was to create a fully functioning automobile repair database. The project aims to achieve these goals:

- Merge vehicle service records, customer information, and repair details into a single centralized database system.
- Develop SQL queries that make it possible to do fast data retrieval, customer lookups, and inventory awareness.
- Generate business intelligence reports that show repair trends, service frequency, and customer behavior.
- Demonstrate how a structured database decreases data redundancy and increases operational accuracy.

These goals aim to create a high quality database which can provide value to auto repair businesses, in both saving time, ease of use, and better decision making.

Problem Statement

Business Problem

Auto repair businesses generate large amounts of service data daily. Each visit produces data from the customer, the vehicle, the type of work, the parts used, and the costs associated with each task. Without a centralized database, all this data is unorganized and difficult to track. This can create five potential problems:

- Fragmented Service History: Staff members cannot retrieve a vehicle's entire history, leading to wasted time for laborers and customers.
- Disconnected Service Records: Customer contact information is not linked to vehicle ownership records, making it difficult to follow up with customers on service reminders.
- No Parts Traceability: Parts usage is not tracked for specific service sessions, leading to inventory planning being reactive.
- Billing Inefficiencies: Without a central record, generating correct bills and finding unpaid accounts can only be done manually.
- Low Business Analytics: Without proper business insights, managers do not have a way to track which services are used the most, which type of vehicles are serviced the most, and how demand shifts according to seasonal changes.

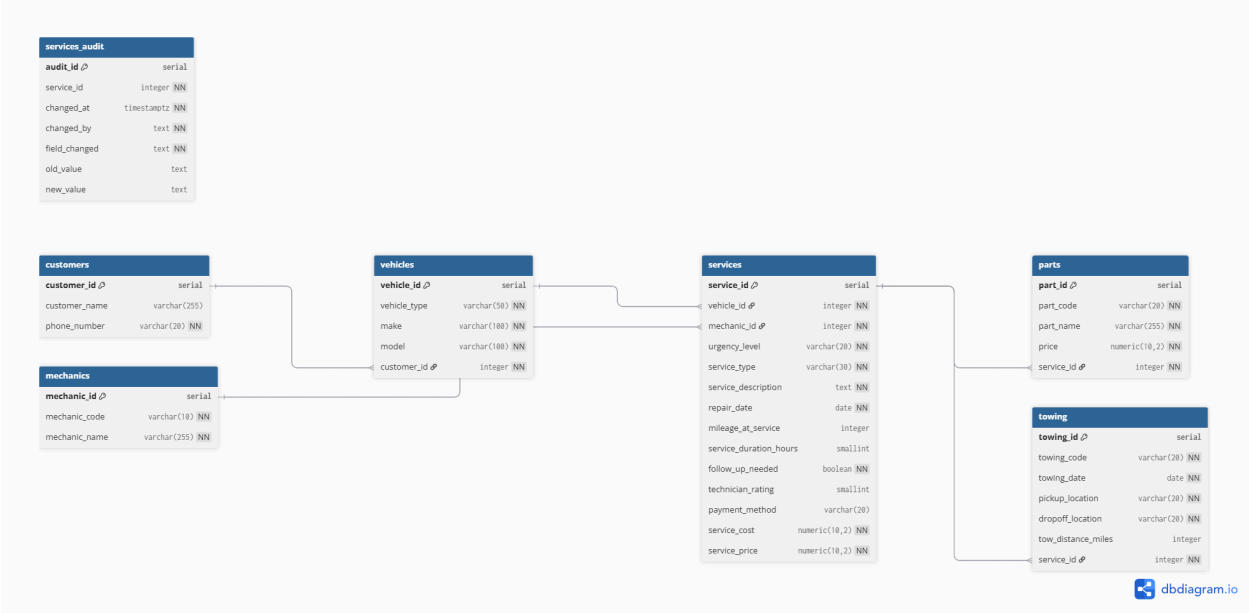
To address these challenges, a structured system that allows quick searches, filtered reports, and overall analysis is necessary. Beyond these issues beyond solved, a centralized database also helps an auto repair shop execute lookups quicker, identify important customers for loyalty programs, anticipate parts needed through past repairs, and generate highly accurate financial reports without manual action. These improvements lead to more productivity, better customer retention, and more informed management choices, and many auto repair shops may not even fully be aware of the potential within all of this.

Database Design

Entity-Relationship Diagram (ERD)

An Entity-Relationship Diagram summarizes the various relations between different data within a database. This is done in a visual manner, which helps the

user see how the various data points work with one another. Below is an ERD that summarizes the various relationships between our main data tables.



This ERD showcases the relational connections between the various tables of the database. Notice the several one-to-many connections between the various tables. This is a key part of the database, due to the nature of how the various data points interact with one another.

Design Justification

This database design directly addresses the auto repair industry's core problem: scattered data across spreadsheets, disconnected records, and a lack of traceability. Consolidating everything into a normalized relational structure ensures data is stored once and referenced everywhere. The SERVICES table acts as the operational hub of the entire system, linking vehicles, mechanics, parts, and towing into a single queryable record, which enables the kind of business reporting that was previously impossible without manual reconciliation.

Microsoft Access Scale Limitations

The original rendition of this project was done through Microsoft Access. The issue was that the original Kaggle dataset, linked here, was extremely large, with well over a million different entries. Microsoft Access struggled with the size of the dataset, so to make it more manageable, the dataset was reduced to around

15,000 entries, but the ultimate decision was to transfer from Microsoft Access to a full PostgreSQL environment. This ultimately allows for a better database with more flexibility.

Conclusion

This project demonstrates how a structured, centralized database can transform the way an auto repair business operates. The five problems outlined at the start are all directly addressed by what was built here. Beyond solving those problems, the PostgreSQL version goes further than the original Microsoft Access database ever could, with enforced data integrity, role-based access control, automated audit logging, and a Python analytics layer that turns raw data into real business insights. The database is built to scale, and future iterations could expand into a front-end interface, a REST API, or direct integration with accounting and scheduling systems. What started as a class project became something that reflects how databases actually work in the real world.